C# اليوم 4 · الأسبوع 1 · ساعتان · دورة أساسيات البرمجة

أول برنامج بـ C#: المتغيرات وأنواع البيانات

أهداف التعلم

في نهاية الدرس يستطيع المتدرب أن:

- ينشئ مشروع Console App في Visual Studio ويشغله.
- يطبع النصوص بـ Console.WriteLine.
- يعرّف متغيرات من الأنواع int وdouble وstring وbool.
- يقرأ مدخلات المستخدم ويحولها إلى أرقام.
- يدمج المتغيرات في النصوص باستخدام $"...".

توزيع وقت الحصة

0:00 – 0:10	مراجعة سريعة لدرس الأمس وحل الواجب
0:10 – 0:50	شرح الدرس مع أمثلة حيّة يكتبها المدرب والمتدربون معًا
0:50 – 1:00	استراحة
1:00 – 1:40	تمارين عملية فردية (المدرب يتنقل بين المتدربين الستة)
1:40 – 1:55	تصحيح جماعي ومناقشة الأخطاء الشائعة
1:55 – 2:00	ملخص الدرس + اختبار ذاتي سريع + الواجب

شرح الدرس

1. إنشاء أول مشروع

افتح **Console App** ← Create a new project ← **Visual Studio Community** (C#) ← اختر أحدث إصدار **.NET.** يظهر في القائمة ← Create. ثم اضغط `F5` أو زر التشغيل الأخضر.

C#

```
// هذا تعليق: لا ينفذه الحاسوب
Console.WriteLine("مرحبًا بالعالم!");
Console.WriteLine("أنا أتعلم C#");
```

```
Output ▸ مرحبًا بالعالم!
أنا أتعلم C#
```

ماذا يفعل هذا الكود؟ برنامج بسيط يطبع سطرين على الشاشة: «مرحبًا بالعالم!» ثم «أنا أتعلم C#». السطر الأول تعليق لا يظهر عند التشغيل.

`// هذا تعليق: لا ينفذه الحاسوب`	الرمزان // يبدآن «تعليقًا»، وهو ملاحظة نكتبها للبشر داخل الكود. الحاسوب يتجاهل كل ما بعد // حتى نهاية السطر، فلا يؤثر على عمل البرنامج.
`Console.WriteLine("مرحبًا بالعالم!");`	Console هي «الشاشة السوداء» التي يكتب فيها البرنامج، و WriteLine أمر يطبع النص الموجود بين القوسين ثم ينتقل إلى سطر جديد. النص يوضع بين علامتي تنصيص "..."، والفاصلة المنقوطة ; في النهاية تعني «انتهت هذه التعليمة» ويجب وضعها بعد كل تعليمة في C#.
`Console.WriteLine("أنا أتعلم C#");`	تعليمة ثانية من نفس النوع تطبع «أنا أتعلم C#» في سطر جديد. الحاسوب ينفذ التعليمات بالترتيب من الأعلى إلى الأسفل، لذلك يظهر هذا السطر بعد الأول.

كل أمر في C# ينتهي بفاصلة منقوطة ; ونسيانها هو أكثر خطأ شائع عند المبتدئين. ولغة C# حساسة لحالة الأحرف: Console غير console.

2. المتغيرات وأنواع البيانات

المتغير صندوق له **نوع** و**اسم** و**قيمة**.

النوع	يخزن	مثال
`int`	عدد صحيح	`int age = 20;`
`double`	عدد عشري	`double price = 9.5;`
`string`	نص	`string name = "Ali";`
`bool`	صح/خطأ	`bool passed = true;`
`char`	حرف واحد	`char grade = 'A';`

C#

```
string name = "سارة";
int age = 22;
double gpa = 3.75;
bool isStudent = true;

Console.WriteLine($"الاسم: {name}");
Console.WriteLine($"العمر: {age} - المعدل: {gpa}");
Console.WriteLine($"طالبة؟ {isStudent}");
```

```
الاسم: سارة ▸ Output
العمر: 22 - المعدل: 3.75
```

```
طالبة؟ True
```

ماذا يفعل هذا الكود؟ ينشئ البرنامج أربعة متغيرات من أنواع مختلفة (نص، عدد صحيح، عدد عشري، صح/خطأ) ثم يطبع قيمها داخل جمل. سيرى الطالب: الاسم: سارة، ثم العمر والمعدل، ثم «طالبة؟ True».

`string name = "سارة";`	هنا ننشئ «متغيرًا»، وهو صندوق في الذاكرة له اسم ويحفظ قيمة. الكلمة string تعني أن نوعه «نص»، واسمه name، والعلامة = تعني «ضع القيمة التي على اليمين داخل المتغير»، فيصبح name يحتوي على النص "سارة".
`int age = 22;`	الكلمة int تعني «عدد صحيح» أي عدد بدون كسور. أنشأنا متغيرًا اسمه age ووضعنا فيه 22، ولاحظ أن الأعداد لا توضع بين علامات تنصيص.
`double gpa = 3.75;`	الكلمة double تعني «عدد عشري» يمكن أن يحتوي على فاصلة عشرية. المتغير gpa يحفظ المعدل 3.75، ونكتب الفاصلة العشرية في الكود بنقطة دائمًا.
`bool isStudent = true;`	الكلمة bool نوع لا يحمل إلا قيمتين: true (صحيح) أو false (خطأ). المتغير isStudent يحفظ true أي «نعم، هي طالبة».
`Console.WriteLine($"الاسم: {name}");`	العلامة $ قبل النص تسمح لنا بوضع متغيرات داخل النص بين قوسين معقوفين { }، وهذا يسمى «إدراج القيم في النص». عند التشغيل يُستبدل {name} بقيمته فيظهر: الاسم: سارة.
`Console.WriteLine($"العمر: {age} - المعدل: {gpa}");`	يمكن إدراج أكثر من متغير في النص نفسه. هنا يُستبدل {age} بـ 22 و {gpa} بـ 3.75، فيظهر العمر والمعدل في سطر واحد.
`Console.WriteLine($"طالبة؟ {isStudent}");`	عند طباعة متغير من نوع bool تكتب C# قيمته بالإنجليزية وبحرف كبير في أوله، فيظهر على الشاشة: طالبة؟ True.

3. قراءة مدخلات المستخدم

`Console.ReadLine()` يقرأ ما يكتبه المستخدم كنص دائمًا. لتحويله إلى رقم نستخدم `int.Parse` أو `double.Parse`.

C#

```
Console.Write("ما اسمك؟ ");
string name = Console.ReadLine();

Console.Write("كم عمرك؟ ");
int age = int.Parse(Console.ReadLine());

Console.WriteLine($"أهلًا {name}، بعد 5 سنوات سيكون عمرك {age + 5}");
```

```
Output ▸ ما اسمك؟ خالد
كم عمرك؟ 20
أهلًا خالد، بعد 5 سنوات سيكون عمرك 25
```

ماذا يفعل هذا الكود؟ يسأل البرنامج المستخدم عن اسمه وعمره، ثم يرحب به ويخبره كم سيكون عمره بعد 5 سنوات. مثلًا إذا كتب «أحمد» و 20 يظهر: أهلًا أحمد، بعد 5 سنوات سيكون عمرك 25.

`Console.Write("ما اسمك؟ ");`	الأمر Write يطبع النص مثل WriteLine لكنه لا ينتقل إلى سطر جديد، فيبقى المؤشر بجانب السؤال ليكتب المستخدم إجابته في نفس السطر.
`string name = Console.ReadLine();`	الأمر Console.ReadLine ينتظر حتى يكتب المستخدم شيئًا ويضغط Enter، ثم يعيد ما كتبه على شكل نص (string). نحفظ هذا النص في المتغير name لنستخدمه لاحقًا.
`Console.Write("كم عمرك؟ ");`	نطبع السؤال الثاني بنفس الطريقة، دون الانتقال إلى سطر جديد.
`int age = int.Parse(Console.ReadLine());`	ReadLine يعيد دائمًا نصًا حتى لو كتب المستخدم أرقامًا، والنص "20" لا يمكن جمعه كعدد. لذلك نستخدم int.Parse الذي يحوّل النص إلى عدد صحيح int، وإذا كتب المستخدم شيئًا ليس عددًا يتوقف البرنامج برسالة خطأ.
`Console.WriteLine($"أهلًا {name}، بعد 5 {age + 5} سنوات سيكون عمرك");`	داخل الأقواس { } في النص الذي يبدأ بـ $ يمكن وضع عملية حسابية وليس متغيرًا فقط. هنا {age + 5} يُحسب أولًا ثم تُطبع النتيجة، أما المتغير age نفسه فلا تتغير قيمته.

لعرض العربية بشكل صحيح في الكونسول أضف في أول البرنامج: `Console.OutputEncoding = System.Text.Encoding.UTF8;`

التمارين العملية

تمرين 1: برنامج ترحيب

سهل

اطلب من المستخدم اسمه ومدينته، ثم اطبع: «مرحبًا [الاسم] من [المدينة]!».

الحل المقترح:

C#

```
Console.Write("اسمك: ");
string name = Console.ReadLine();
Console.Write("مدينتك: ");
string city = Console.ReadLine();
Console.WriteLine($"مرحبًا {name} من {city}!");
```

ماذا يفعل هذا الكود؟ يسأل البرنامج المستخدم عن اسمه ومدينته ثم يطبع ترحيبًا يجمعهما، مثل: مرحبًا سارة من مسقط!

`Console.Write("اسمك: ");`	Console.Write يطبع السؤال ويترك المؤشر في نفس السطر حتى يكتب المستخدم بجانبه.
`string name = Console.ReadLine();`	Console.ReadLine يقرأ ما يكتبه المستخدم حتى يضغط Enter ويعيده كنص، فنحفظه في المتغير name من نوع string.
`Console.Write("مدينتك: ");`	نطبع السؤال الثاني عن المدينة بنفس الطريقة.
`string city = Console.ReadLine();`	نقرأ اسم المدينة ونحفظه في متغير نصي جديد اسمه city. لا نحتاج إلى تحويل هنا لأننا نريد نصًا أصلًا.
`Console.WriteLine($"مرحبًا {name} من {city}!");`	النص الذي يبدأ بـ $ يسمح بإدراج المتغيرات بين { }، فيُستبدل {name} و {city} بما كتبه المستخدم ثم يُطبع السطر كاملًا.

تمرين 2: مساحة المستطيل

متوسط

اقرأ الطول والعرض (أعداد عشرية) واطبع المساحة والمحيط.

الحل المقترح:

C#

```
Console.Write("الطول: ");
double length = double.Parse(Console.ReadLine());
Console.Write("العرض: ");
double width = double.Parse(Console.ReadLine());

double area = length * width;
double perimeter = 2 * (length + width);
Console.WriteLine($"المساحة = {area}");
Console.WriteLine($"المحيط = {perimeter}");
```

ماذا يفعل هذا الكود؟ يطلب البرنامج طول المستطيل وعرضه، ثم يحسب مساحته ومحيطه ويطبعهما. مثلًا مع طول 5 وعرض 3 يظهر: المساحة = 15 ثم المحيط = 16.

`Console.Write("الطول: ");`	نطبع طلبًا للمستخدم ليكتب الطول، ويبقى المؤشر في نفس السطر لأننا استخدمنا Write وليس WriteLine.
`double length = double.Parse(Console.ReadLine());`	ReadLine يعيد نصًا، و double.Parse يحوّله إلى عدد عشري double حتى نستطيع إجراء الحساب عليه، ويقبل أعدادًا بكسور مثل 2.5. نحفظ النتيجة في المتغير length.
`Console.Write("العرض: ");`	نطلب من المستخدم إدخال العرض.

`double width = double.Parse(Console.ReadLine());`	نقرأ العرض ونحوّله إلى عدد عشري بنفس الطريقة ونحفظه في width.
`double area = length * width;`	العلامة * تعني الضرب في C#. نضرب الطول في العرض للحصول على المساحة ونحفظ الناتج في متغير جديد اسمه area.
`double perimeter = 2 * (length + width);`	الأقواس ( ) تجعل الجمع يُنفذ أولًا، ثم يُضرب الناتج في 2. بدون الأقواس كانت C# ستضرب 2 في length أولًا لأن الضرب يسبق الجمع، فتكون النتيجة خاطئة.
`Console.WriteLine($"المساحة = {area}");`	نطبع المساحة بإدراج المتغير area داخل النص باستخدام $ والأقواس { }.
`Console.WriteLine($"المحيط = {perimeter}");`	نطبع المحيط في سطر جديد بنفس الطريقة.

تمرين 3: تحدٍّ: تحويل الحرارة

تحدٍّ

اقرأ درجة الحرارة بالمئوية وحوّلها إلى فهرنهايت بالمعادلة F = C × 9 / 5 + 32.

الحل المقترح:

C#

```
Console.Write("الحرارة بالمئوية: ");
double c = double.Parse(Console.ReadLine());
double f = c * 9 / 5 + 32;
Console.WriteLine($"{c}°C = {f}°F");
```

```
الحرارة بالمئوية: 30
30°C = 86°F
```

ماذا يفعل هذا الكود؟ يطلب البرنامج درجة الحرارة بالمئوية ويحوّلها إلى فهرنهايت. مثلًا إذا أدخل المستخدم 100 يظهر: 100°C = 212°F.

`Console.Write("الحرارة بالمئوية: ");`	نطبع طلبًا للمستخدم ليكتب درجة الحرارة، ويبقى المؤشر بجانبه لأننا استخدمنا Write.
`double c = double.Parse(Console.ReadLine());`	نقرأ ما كتبه المستخدم كنص بـ ReadLine ثم نحوّله إلى عدد عشري بـ double.Parse، ونحفظه في المتغير c.

`double f = c * 9 / 5 + 32;`	هذه معادلة التحويل: C# تنفذ الضرب * والقسمة / أولًا من اليسار إلى اليمين، ثم الجمع + في الأخير. ولأن c من نوع double فالنتيجة تحتفظ بالكسور ولا تضيع.
`Console.WriteLine($"{c}°C = {f}°F");`	نطبع القيمتين داخل نص واحد باستخدام $ و { }، والرموز °C و °F مجرد نص عادي يُطبع كما هو.

اختبر نفسك (3 أسئلة)

1. أي نوع يناسب تخزين السعر 12.5؟

int · double · bool

2. ماذا يُرجع Console.ReadLine()؟

رقمًا · نصًا string · قيمة bool

3. ما الذي ينقص هذا السطر؟ int x = 5

فاصلة منقوطة ; · قوسان () · علامة $

الإجابات: 1) double · 2) نصًا string · 3) فاصلة منقوطة ;

الواجب: اكتب برنامجًا يقرأ اسم منتج وسعره وكميته ويطبع الإجمالي.